

Tập luận văn đội tuyển quốc gia Trung Quốc năm 2015

Huấn luyện viên: Yu Linyun

China Computer Federation, 2015

Nguồn gốc: Tập luận văn ứng viên đội tuyển quốc gia Trung Quốc Olympic Tin học năm 2015

IOI China candidate team papers / National training team 2015 collection.pdf

Tóm tắt

PDF nguồn là tập hợp luận văn đội tuyển quốc gia Trung Quốc năm 2015. Tập được tạo bằng pdfTeX nhưng lớp văn bản không ánh xạ đúng về Unicode: ngay cả mục lục khi trích xuất bằng pdftotext cũng trở thành mojibake. Bản dịch này chuyển ngữ phân đầu sách, mục lục đọc được từ các trang đã render, và hai bài đầu tiên của tập sách, được OCR từ ảnh trang.

1 Thông tin đầu sách

Tập sách có tiêu đề *Tập luận văn ứng viên đội tuyển quốc gia Trung Quốc Olympic Tin học năm 2015*. Huấn luyện viên ghi trên bìa là Yu Linyun. Đơn vị xuất bản là China Computer Federation.

Ghi chú trích xuất. Do lớp văn bản của PDF nguồn không dùng được, các bài đã dịch được đọc từ ảnh render bằng pdftoppm và OCR bằng tesseract -l chi_sim+eng, sau đó đối chiếu trực quan ở các trang biên. Hiện bản dịch đã hoàn thành hai bài đầu tiên: bài “Mở rộng suffix automaton trên trie” dùng các trang PDF vật lý 5–20, tương ứng trang in 1–16; bài “Bàn về ứng dụng tư tưởng heuristic trong thi tin học” dùng các trang PDF vật lý 21–40, tương ứng trang in 17–36. Trang PDF vật lý 41, trang in 37, là đầu bài kế tiếp về khớp chuỗi.

2 Mục lục đã dịch

Trang	Bài viết	Tác giả / trường
1	Mở rộng suffix automaton trên trie	Liu Yanyi, Trường Yali, Trường Sa, Hồ Nam
17	Bàn về ứng dụng tư tưởng heuristic trong thi tin học	Ren Zhizhou, Trường Trung học số 1 Thiệu Hưng
37	Bàn về một số phương pháp khớp chuỗi	Wang Shenghao, Trường Trung học số 1 Thiệu Hưng
67	Suffix automaton và ứng dụng	Zhang Tianyang, Trường Trung học số 1 Trường Sa, Hồ Nam
81	Phép toán trên hàm sinh và bài toán đếm tổ hợp	Jin Ce, Trường Học Quân Hàng Châu, Chiết Giang
103	Báo cáo ra đề <i>yc's bonus</i>	Liu Jiancheng, Trường Yali, Trường Sa
113	Bàn về ứng dụng chia khối trong một lớp bài toán trực tuyến	Zou Daoyao, Trường Trung học Trần Hải, Ninh Ba
129	Thuật toán liên quan tới cactus và ứng dụng	Wang Yisong, Trường Học Quân Hàng Châu, Chiết Giang
149	Bàn về thuật toán matching trên đồ thị và ứng dụng	Chen Haobo, Trường Trường Quận, Trường Sa, Hồ Nam
175	Bàn về các bài toán vật lý trong thi tin học	Chen Siyu, Trường Trung học số 2 Hàng Châu
199	Báo cáo ra đề <i>Đề bài bị thất lạc</i>	Yu Jiping, Trường Trung học số 24 Đại Liên
205	Một số kỹ thuật tối ưu DP	Zhang Hengjie, Trường Trung học số 1 Thiệu Hưng
217	Báo cáo ra đề <i>Product</i>	Du Yuhao, Trường Trung học Trần Hải, Ninh Ba
231	Bước đầu tìm hiểu một lớp bài toán lấy mã nguồn làm đầu vào	Lu Xiaochen, Học viện Giao Xuyên Trần Hải, Ninh Ba
271	Tính chất, ứng dụng và thuật toán nhanh cho lũy thừa tập hợp	Lu Kaifeng, Trường Trung học số 2 Vũ Hán, Hồ Bắc

3 Mở rộng suffix automaton trên trie

Liu Yanyi

Trường Yali, Trường Sa, Hồ Nam

Tóm tắt

Bài viết phân tích chi tiết một automaton hữu hạn xác định có thể nhận ra mọi hậu tố của một trie. Tác giả đưa ra và chứng minh các kết luận rằng số trạng thái là tuyến tính, tổng số hàm chuyển không vượt quá số nút của trie nhân với kích thước bảng chữ cái, đồng thời trình bày một thuật toán xây dựng khả thi, có độ phức tạp tuyến tính khi được phép xử lý offline. Bài viết cũng phân tích độ phức tạp của thuật toán xây dựng online, giới thiệu một số ứng dụng liên quan của suffix automaton, với hy vọng gợi mở thêm

các hướng sử dụng cấu trúc này.

3.1 Lời nói đầu

Trong thi thuật toán, xử lý xâu là một mảng rất quan trọng. Nhìn chung có hai loại bài toán lớn: bài toán có mẫu cố định và bài toán có văn bản chính cố định. Khi mẫu cố định, có thể có nhiều văn bản chính khác nhau. Nếu chỉ có một mẫu, thuật toán KMP nổi tiếng [2, 3] xử lý được lớp bài toán này; cũng có thể dùng hashing xâu [13] để đếm số lần xuất hiện của mẫu. Nếu có nhiều mẫu, thuật toán AC automaton quen thuộc [1] đảm nhiệm tốt lớp bài toán đó. Nếu văn bản chính cố định, ta thường xây suffix array [11], suffix tree [4, 5] hoặc suffix automaton [7] cho văn bản chính. Lớp bài toán này đã được nghiên cứu đầy đủ trong cộng đồng thi thuật toán. Chẳng hạn, Chen Lijie đã nêu nhiều ứng dụng của suffix automaton trên một xâu trong bài giảng giao lưu trại đông [8], còn Luo Suijian giải thích chi tiết việc dùng suffix array trong thi tin học trong luận văn đội tuyển quốc gia của mình [12]. Khi đề bài cho phép xử lý offline, hai loại bài toán trên thường có thể chuyển đổi cho nhau, thể hiện tính linh hoạt của các thuật toán xử lý xâu.

Trong kỳ chọn đội tuyển tỉnh Chiết Giang năm 2015 xuất hiện bài toán sau: cho một cây có $n \leq 10^5$ nút, số lá không quá 20, mỗi cạnh mang một ký tự. Đường đi giữa hai nút trên cây tạo thành một xâu; hỏi có tất cả bao nhiêu xâu con khác nhau.¹ Bài này rõ ràng thuộc loại văn bản chính cố định, nhưng văn bản chính không còn là một xâu mà là một cây. Nếu trực tiếp liệt kê mọi cặp đỉnh rồi dùng trie [1, 13] để kiểm tra hai xâu có bằng nhau hay không, độ phức tạp là $O(n^3)$. Ngay cả dùng hashing xâu [13], thời gian vẫn là $O(n^2)$, không thể qua được dữ liệu.

Ta chuyển hóa bài toán như sau. Vì chỉ có 20 lá, với mỗi lá làm gốc, dựng một trie riêng rồi hợp nhất tất cả các trie này thành một trie mới. Khi đó mọi xâu xuất hiện trên cây ban đầu chắc chắn xuất hiện trong trie mới; văn bản chính đã trở thành một trie.

Để giải quyết vấn đề trên, bài viết phân tích automaton hữu hạn xác định nhận ra toàn bộ hậu tố của một trie và đề xuất một cách xây dựng. Nội dung chính gồm năm phần:

1. đưa một số khái niệm trên xâu sang trie, đồng thời giới thiệu suffix automaton của trie;
2. phân tích suffix link và xây dựng cấu trúc cây parent;
3. chứng minh số trạng thái là tuyến tính và nêu chặn trên cho tổng số hàm chuyển trong automaton;
4. mô tả thuật toán xây suffix automaton cho trie; khi được phép offline, độ phức tạp của nó là tuyến tính theo kích thước automaton;
5. trình bày một số ứng dụng của suffix automaton trên trie, qua đó làm rõ hơn cách nhìn về suffix automaton.

3.2 Suffix automaton của trie

Trong bài viết này, mọi ký tự của xâu và mọi ký tự trên cạnh của trie đều thuộc một bảng chữ cái hữu hạn A . Tập mọi xâu sinh bởi bảng chữ cái này là A^* ; xâu rỗng được ký hiệu là ε . Tập không chứa xâu rỗng là $A^+ = A^* \setminus \{\varepsilon\}$. Dùng $|s|$ để chỉ độ dài xâu s , s_i là ký tự thứ i , và $s_{i,j}$ là xâu gồm các ký tự từ vị trí i đến vị trí j của s . Ký hiệu $a \cdot b$ là xâu thu được khi nối a với b ; dấu nhân có thể được lược bỏ.

Tập mọi hậu tố của một xâu s là

$$S(s) = \{s_{i,|s|} \mid 1 \leq i \leq |s|\}.$$

Với một tập xâu S , $\text{Minl}(S)$ và $\text{Maxl}(S)$ lần lượt là độ dài của xâu ngắn nhất và dài nhất trong S .

¹ZJOI 2015 Day 1, *Vùng đất huyền tưởng được chut thần ưu ái*, tác giả Chen Lijie.

Dùng T để chỉ một trie, và dùng $T_x, T_y, T_z, \dots$ để chỉ các nút của T . Xây nhận được bằng cách nối theo thứ tự các ký tự trên đường đi từ nút T_x đến một nút T_y trong cây con của nó được ký hiệu là $T_{x,y}$. Nếu không nói rõ, khi dùng $T_{x,y}$, ta luôn ngầm hiểu T_y nằm trong cây con của T_x .

Một tiền tố của T là

$$P_{T_x} = T_{\text{root},x} \quad (\text{root là gốc của } T, T_x \in T).$$

Tập mọi xâu con của T là

$$F(T) = \{T_{x,y} \mid T_x, T_y \in T, T_y \text{ nằm trong cây con của } T_x\}.$$

Tương tự, tập mọi hậu tố của T là

$$S(T) = \{T_{x,y} \mid T_x, T_y \in T, T_y \text{ là nút lá}\}.$$

Xây suffix automaton cho trie nghĩa là xây một automaton hữu hạn có thể nhận ra mọi xâu trong $S(T)$, đồng thời cố gắng giảm số trạng thái và số chuyển xuống càng nhỏ càng tốt. Tương tự suffix automaton trên một xâu, ta định nghĩa vị trí xuất hiện của một xâu trên trie:

$$\text{Right}(s) = \{T_y \mid T_y \in T, \exists T_x \in T, T_y \text{ nằm trong cây con của } T_x, T_{x,y} = s\}.$$

Định nghĩa quan hệ tương đương R_T ; khi không gây nhầm lẫn viết là R :

$$aRb \iff \text{Right}(a) = \text{Right}(b) \quad (a, b \in A^*).$$

Gọi $R(s)$ là tập mọi xâu tương đương với s , tức một lớp tương đương. Trong automaton, mỗi trạng thái tương ứng với một lớp tương đương.

Gọi suffix automaton xây từ T là $\Phi(T)$, trạng thái đầu là $\Phi(T)_\varepsilon$. Ký hiệu $\Phi(T)_s$, với $s \in F(T)$, là trạng thái sau khi đọc s . Hàm chuyển được định nghĩa bởi chuyển nhân a từ $\Phi(T)_s$ tới $\Phi(T)_{sa}$ khi $a \in A$ và $sa \in F(T)$. Khi nối hai xâu s_1, s_2 mà $s_1, s_2, s_1s_2 \in F(T)$, ta cũng có $\Phi(T)_{s_1}$ đọc s_2 tới $\Phi(T)_{s_1s_2}$. Theo định nghĩa,

$$\Phi(T)_a = \Phi(T)_b \iff aRb \quad (a, b \in F(T)).$$

Nếu a là một xâu tương ứng với trạng thái $\Phi(T)_b$, thì tập xâu tương ứng với trạng thái $\Phi(T)_b$ chính là $R(b)$.

Hình a. Trong trie minh họa của bản gốc, b tương đương với ab ; các nút màu đỏ là tập Right của chúng.

Để phân tích số trạng thái của toàn bộ cấu trúc, tức số lớp tương đương R trong $F(T)$, ta cần xét một số tính chất của các trạng thái suffix automaton. Mục tiêu là làm số trạng thái nhỏ nhất có thể.

Bổ đề 2.1. Với $x, y, z, u \in F(T)$, ta có:

- (i) $xRy \Rightarrow (x \in S(y) \text{ hoặc } y \in S(x))$,
- (ii) $x \in S(z) \Rightarrow \text{Right}(z) \subseteq \text{Right}(x)$,
- (iii) $(x \in S(z), z \in S(u), xRu) \Rightarrow zRuRx$.

Chứng minh. (i) Không mất tính tổng quát, giả sử $|x| \leq |y|$ và $Right(x) = Right(y)$. Lấy tùy ý $T_v \in Right(x)$. Khi đó $x \in S(P_{T_v})$ và $y \in S(P_{T_v})$, nên $x \in S(y)$.

(ii) Với mọi $T_v \in Right(z)$, ta có $z \in S(P_{T_v})$. Vì $x \in S(z)$, suy ra $x \in S(P_{T_v})$, do đó $T_v \in Right(x)$. Vậy $Right(z) \subseteq Right(x)$.

(iii) Từ (ii), $z \in S(u)$ cho $Right(u) \subseteq Right(z)$, và $x \in S(z)$ cho $Right(z) \subseteq Right(x)$. Lại có xRu , nên $Right(z) = Right(u) = Right(x)$, tức $zRuRx$.

Mệnh đề (i) cũng cho thấy trong cùng một lớp tương đương R , không thể có hai xâu khác nhau nhưng cùng độ dài.

Hệ quả 2.2. Với một lớp tương đương $R(a)$, gọi x là xâu dài nhất trong $R(a)$, và y là xâu ngắn nhất trong $R(a)$. Khi đó

$$R(a) = \{z \mid z \in S(x) \text{ và } |y| \leq |z| \leq |x|\}.$$

Điều này suy ra trực tiếp từ Bổ đề 2.1.

3.3 Suffix link

Trong thuật toán xây suffix automaton cho một xâu, suffix link giữ vai trò then chốt [7, 6], và toàn bộ thuật toán rất giống KMP [3]. Khi xây suffix automaton cho trie, ta cũng giữ suffix link và so sánh quá trình xây dựng với thuật toán xây automaton tối thiểu xác định nhận ra tập $A^*\{P_{T_x}\}$, tức cách xây AC automaton quen thuộc [1]. Tương tự hàm fail trong AC automaton, ta định nghĩa suffix link của suffix automaton trên trie như sau.

Suffix link là một hàm $s_T : A^* \setminus \{\varepsilon\} \rightarrow A^*$, khi không nhầm lẫn viết là s :

$$s_T(x) = \{z \mid z \in S(x), |z| = \text{Minl}(R_T(x)) - 1\}.$$

Nói cách khác, đó là hậu tố dài nhất y của x sao cho xR_Ty không đúng.

Bổ đề 3.1. Với $x, y \in F(T)$, ta có:

- (i) $xRy \Rightarrow s(x) = s(y)$,
- (ii) $Right(x) \subset Right(s(x))$,
- (iii) $\text{Minl}(R(x)) - 1 = |s(x)| = \text{Maxl}(R(s(x)))$.

Chứng minh. (i) Không mất tính tổng quát, giả sử $|x| \leq |y|$. Theo định nghĩa, $|s(x)| = \text{Minl}(R(x)) - 1$, $|s(y)| = \text{Minl}(R(y)) - 1$, mà $R(x) = R(y)$, nên $|s(x)| = |s(y)| < |x| \leq |y|$. Theo Bổ đề 2.1(i), $x \in S(y)$. Lại có $s(x) \in S(x)$ và $s(y) \in S(y)$, do đó $s(x) = s(y)$.

(ii) Theo Bổ đề 2.1(ii), $Right(x) \subseteq Right(s(x))$. Vì $xRs(x)$ là sai, hai tập này không bằng nhau, nên bao hàm là nghiêm ngặt.

(iii) Vì $s(x) \in R(s(x))$, ta có $|s(x)| \leq \text{Maxl}(R(s(x)))$. Giả sử tồn tại xâu y tương đương với $s(x)$ và dài nhất trong lớp đó. Khi ấy $Right(y) = Right(s(x))$, đồng thời $s(x) \in S(y)$. Dùng phản chứng: nếu $|y| > |s(x)|$, lấy $T_v \in Right(x)$. Từ (ii), $T_v \in Right(s(x))$, suy ra $T_v \in Right(y)$, nên $y \in S(P_{T_v})$ và $x \in S(P_{T_v})$. Nếu $|y| > |x|$ thì $x \in S(y)$, theo Bổ đề 2.1(ii) sẽ có $Right(s(x)) = Right(y) \subseteq Right(x)$, mâu thuẫn với (ii). Nếu $|y| \leq |x|$, vì xRy không đúng nên $s(x) = y$, mâu thuẫn với $|y| > |s(x)|$. Vậy mệnh đề được chứng minh.

Từ (i), mọi xâu trong cùng một lớp tương đương $R(a)$ có suffix link trở tới cùng một xâu. Vì vậy ta có thể gán suffix link cho từng trạng thái của suffix automaton. Cụ thể, suffix link của $R(a)$ trở tới $R(s(a))$.

Theo (ii) hoặc (iii), các suffix link này tạo thành một cây; ta gọi nó là **cây parent**. Định nghĩa $Fa(\Phi(T)_s)$ là suffix link của trạng thái $\Phi(T)_s$, tức

$$Fa(\Phi(T)_a) = \Phi(T)_b \iff s(a)Rb.$$

Bổ đề 3.2. Với $x \in F(T)$, lấy tùy ý $T_v \in \text{Right}(x)$. Nếu $|P_{T_v}| > \text{Maxl}(R(x))$ và đặt $a = P_{T_v}$, thì

$$s(a_{|a|-\text{Maxl}(R(x)), |a|})Rx.$$

Chứng minh. Đặt $b = a_{|a|-\text{Maxl}(R(x)), |a|}$. Từ giả thiết có $x \in S(b)$. Theo Hệ quả 2.2, vì $|b| > \text{Maxl}(R(x))$, nên bRx không đúng, còn hậu tố độ dài $\text{Maxl}(R(x))$ của b lại tương đương với x . Vì vậy b không tương đương với hậu tố đó; theo định nghĩa suffix link, $s(b)$ chính là hậu tố này, và $s(b)Rx$.

Hình b. Cây trong bản gốc là cây parent của suffix automaton được xây từ trie ở Hình a.

3.4 Số trạng thái tuyến tính

Với suffix automaton xây cho một xâu x , kích thước automaton là $O(|x|)$ [7, 6]. Ta cũng mong suffix automaton xây cho trie có quy mô trạng thái tuyến tính. Sau khi định nghĩa suffix link, cần phân tích kỹ hơn $\Phi(T)$.

Bổ đề 4.1. Với $x, y \in F(T)$, ta có:

- (i) $\forall y, \neg(xRs(y)) \Rightarrow |\text{Right}(x)| = 1$,
- (ii) $|\text{Right}(x)| = 1 \Rightarrow P_{T_v}Rx \quad (\text{Right}(x) = \{T_v\})$,
- (iii) $\forall T_v \in \text{Right}(x), \neg(xRP_{T_v})$
 $\Rightarrow \exists u, v, s(u)Rs(v)Rx \text{ và } \neg(uRv).$

Chứng minh. (i) Chứng minh bằng phản chứng. Giả sử $|\text{Right}(x)| > 1$. Lấy $T_{v_1}, T_{v_2} \in \text{Right}(x)$ và đặt $a = P_{T_{v_1}}, b = P_{T_{v_2}}$. Khi đó $|a|, |b| \geq \text{Maxl}(R(x))$; nếu một trong hai độ dài nhỏ hơn $\text{Maxl}(R(x))$, xâu trong $R(x)$ có độ dài $\text{Maxl}(R(x))$ không thể xuất hiện ở nút tương ứng, mâu thuẫn. Nếu $a \notin R(x)$, theo Bổ đề 3.2 suy ra suffix link của một hậu tố thích hợp của a trở vào lớp của x . Nếu $a \in R(x)$, vì $T_{v_1} \neq T_{v_2}$, ta có $a \neq b$, và $|b| \geq \text{Maxl}(R(x))$, nên một lập luận tương tự cũng cho một xâu có suffix link trở vào $R(x)$. Mệnh đề được chứng minh.

(ii) Biết $x \in S(P_{T_v})$, do đó $|\text{Right}(P_{T_v})| \leq |\text{Right}(x)|$. Vì $|\text{Right}(x)| = 1$, suy ra $\text{Right}(P_{T_v}) = \{T_v\} = \text{Right}(x)$, tức $P_{T_v}Rx$.

(iii) Từ phản đảo của (i), nếu $|\text{Right}(x)| \neq 1$ thì $|\text{Right}(x)| \geq 2$. Theo giả thiết, với mọi $T_v \in \text{Right}(x)$ ta có $|P_{T_v}| > \text{Maxl}(R(x))$. Tồn tại T_{v_1}, T_{v_2} sao cho hậu tố chung dài nhất của $P_{T_{v_1}}$ và $P_{T_{v_2}}$ có độ dài đúng bằng $\text{Maxl}(R(x))$. Nếu không, mọi cặp đều có hậu tố chung dài hơn $\text{Maxl}(R(x))$; theo Bổ đề 2.1(iii), hậu tố chung dài hơn đó cũng tương đương với x , mâu thuẫn với định nghĩa $\text{Maxl}(R(x))$.

Đặt

$$a = (P_{T_{v_1}})_{|P_{T_{v_1}}|-\text{Maxl}(R(x)), |P_{T_{v_1}}|}, \quad b = (P_{T_{v_2}})_{|P_{T_{v_2}}|-\text{Maxl}(R(x)), |P_{T_{v_2}}|}.$$

Từ cách chọn, $a \neq b$, $a \notin S(b)$, $b \notin S(a)$. Theo Bổ đề 2.1(i), $\neg(aRb)$. Dùng Bổ đề 3.2 ta có $s(a)Rx$ và $s(b)Rx$. Lấy $u = a$, $v = b$ là được.

Định lý 4.2. Suffix automaton xây cho một trie có số trạng thái tuyến tính.

Chứng minh. Xét cây parent tạo bởi suffix link. Từ Bổ đề 4.1(i) và (ii), với mọi lá $\Phi(T)_s$ trên cây parent đều tồn tại một tiền tố P_{T_v} sao cho $P_{T_v}Rs$. Từ Bổ đề 4.1(iii), mọi trạng thái $\Phi(T)_s$ mà với mọi T_v đều không có sRP_{T_v} thì phải có ít nhất hai con trong cây parent. Nói cách khác, các lá và các nút chỉ có một con trên cây parent đều tương ứng với một tiền tố nào đó P_{T_v} ; các nút còn lại có ít nhất hai con.

Số tiền tố chỉ là $|T|$, và trạng thái đạt được khi đọc từng P_{T_v} là duy nhất. Vì vậy tổng số lá và nút một con trên cây parent không vượt quá $|T|$; toàn bộ số nút của cây không vượt quá $2|T|$. Do đó số trạng thái là tuyến tính.

Nếu không chỉ số trạng thái mà tổng số hàm chuyển, tức số cạnh, cũng tuyến tính, thì toàn bộ automaton sẽ tuyến tính. Tuy nhiên, khác với suffix automaton của một xâu, tổng số hàm chuyển của suffix automaton trên trie có thể ở cấp số trạng thái nhân với kích thước bảng chữ cái. Ta chứng minh nó không vượt quá cấp này, và đưa ra một trie đạt cùng cấp độ.

Định lý 4.3. Với một trie, chặn trên của số hàm chuyển trong suffix automaton là

$$O((\text{số nút của trie}) \cdot (\text{kích thước bảng chữ cái})).$$

Chứng minh. Gọi trie là T . Theo Định lý 4.2, $|\Phi(T)| \leq 2|T|$. Với mỗi trạng thái $\Phi(T)_s$, số chuyển nhiều nhất là $|A|$. Vì vậy tổng số hàm chuyển không vượt quá $2|T||A|$.

Ta cũng dựng được một trie khiến số hàm chuyển đạt chặn này theo cấp độ. Chỉ cần xét những ký tự thực sự xuất hiện trên cạnh của T , nên $|A| \leq |T|$. Một phần của T là một dây dài $|T|/2$, mọi cạnh trên dây đều mang cùng ký tự a . Các nút còn lại nối từ nút cuối của dây bằng các ký tự đôi một khác nhau; gọi tập ký tự này là B . Khi đó $a^ib \in S(T)$ với $0 \leq i \leq |T|/2$ và $b \in B$, đồng thời hiển nhiên $\neg(a^iRa^j)$ nếu $i \neq j$. Vì thế với mỗi trạng thái $\Phi(T)_{a^i}$, mọi $b \in B$ đều tạo một hàm chuyển. Tổng số hàm chuyển đạt $|T|/2 \cdot |B|$, tức cùng cấp với $|T|/2 \cdot |A|/2$, trùng cấp với chặn trên trong Định lý 4.3.

Hình c. Ví dụ trie trong bản gốc gồm một dây toàn ký tự a , rồi tỏa ra nhiều cạnh ký tự khác nhau, làm tổng số hàm chuyển đạt chặn trên theo cấp độ.

Dù số chuyển khi xây suffix automaton cho trie không luôn tuyến tính, trong phần lớn đề thi và bài toán thực tế, bảng chữ cái không lớn; thường chỉ là chữ cái Latin thường hoặc chữ số. Khi bảng chữ cái là hằng số, số chuyển vẫn tuyến tính, nên đây vẫn là một phương pháp hiệu quả để xử lý các bài toán xâu liên quan đến trie.

3.5 Thuật toán xây dựng

Trong thuật toán xây suffix automaton cho xâu [7, 8], ta dùng một thuật toán tăng dần online: từ suffix automaton của s , xây suffix automaton của sa , với $a \in A$. Khi xây cho trie, ta cũng xét thuật toán tăng dần online. Từ phân tích số trạng thái và số chuyển ở trên, ta biết cận dưới độ phức tạp của thuật toán xây dựng là $|T||A|$.

Giả sử từ trie T , ta thêm vào nút T_v một cạnh nhãn a đi tới nút con mới T_w , thu được trie mới T_N . Đặt $s = P_{T_v}$, nên tiền tố mới là $sa = P_{T_w}$. Xét những phần của $\Phi(T_N)$ cần sửa trên nền $\Phi(T)$. Trước hết phân tích số trạng thái.

3.5.1 Thay đổi trạng thái

Với mọi xâu đã có thể tiếp tục mở rộng thành hậu tố của trie mới, trạng thái tương ứng trong $\Phi(T_N)$ phải tồn tại. Ngược lại, nếu một xâu không thể mở rộng thành hậu tố mới, trạng thái đó không cần xuất hiện thêm. Nếu một lớp tương đương không đổi sau khi thêm T_w , trạng thái suffix automaton tương ứng cũng không đổi, tức $\Phi(T_N)_x = \Phi(T)_x$.

Trường hợp 1. Nếu T_v đã có một con nối bằng cạnh nhãn a , ta không cần sửa $\Phi(T)$.

Trường hợp 2. Nếu $sa \in F(T)$, nghĩa là trong $\Phi(T)$ đã có chuyển nhãn a từ trạng thái của s , thì lớp tương đương $R_T(sa)$ có thể bị tách làm hai lớp.

Thật vậy, với $x, y \in F(T)$ và $xR_T y$, nếu trong tập $Right_{T_N}$ của x và y đều xuất hiện nút mới T_w , thì khi bỏ T_w ra, hai tập vẫn bằng nhau, nên chắc chắn $xR_{T_N} y$. Vì vậy mọi xâu tương đương với sa trong T_N vốn đều nằm trong cùng lớp $R_T(sa)$. Với $x \in R_T(sa)$, nếu $xR_{T_N} sa$, không có trạng thái mới. Nếu tồn tại $x \in R_T(sa)$ nhưng $\neg(xR_{T_N} sa)$, cần tách $R_T(sa)$ thành hai lớp: một lớp tương đương với sa trong T_N , một lớp không tương đương với nó. Lớp không tương đương có tập $Right$ không chứa T_w , nên tập $Right$ của nó giữ nguyên như trong T .

Xét suffix link của hai trạng thái mới. Gọi $u, v \in F(T)$, trong đó $uR_{T_N} sa$, $\neg(vR_{T_N} sa)$, nhưng $uR_T sa$ và $vR_T sa$. Nếu đặt u vào lớp thứ nhất và v vào lớp thứ hai, vì $sa = P_{T_w}$ và $uR_{T_N} sa$, mọi hậu tố của u nhận thêm cùng vị trí mới nên suffix link của lớp thứ nhất không đổi. Với v , phần mới bị tách ra đúng tại lớp của sa , do đó suffix link của lớp thứ hai trở tới lớp mới chứa sa . Nói ngắn gọn, nếu một trạng thái bị tách, một bản sao giữ suffix link cũ, bản sao còn lại có suffix link trở tới trạng thái của sa .

Hình d. Suffix automaton của trie trong Hình a.

Hình e, f, g. Sau khi thêm một nút vào trie của Hình a, bản gốc minh họa lần lượt trie mới, suffix automaton mới và cây parent mới. Nút và cạnh màu cam biểu thị phần mới trong automaton/cây parent.

Khi đã biết lớp tương đương chứa sa trong T_N , mọi hậu tố của sa trước đó đã được đưa vào các lớp liên quan, và các trạng thái ấy không đổi, nên không cần xét lại.

Trường hợp 3. Nếu $sa \notin F(T)$, ta tạo một lớp mới $R_{T_N}(sa)$. Vì xuất hiện một hậu tố mới, vẫn có khả năng một lớp tương đương cũ bị tách.

Có một kết luận khá rõ ràng: $s_{T_N}(sa)$ bằng hậu tố dài nhất của sa có xuất hiện trong $F(T)$. Chứng minh đơn giản: nếu $x \in S(sa)$ nhưng $x \notin F(T)$, thì x chỉ xuất hiện tại nút mới T_w , và sa cũng chỉ xuất hiện tại T_w ; do đó $Right(sa) = Right(x) = \{T_w\}$, nên $xRsa$. Ngược lại, nếu $x \in S(sa)$ và $x \in F(T)$, thì $|Right(x)| > 1$, nên $\neg(xRsa)$.

Kết luận này giúp quá trình xây dựng thuận tiện hơn. Các hậu tố

$$y \in S(sa), \quad y \notin S(s_{T_N}(sa))$$

tạo thành lớp mới $R_{T_N}(sa)$, và ta tạo một trạng thái mới cho lớp đó. Với những $y \in S(s_{T_N}(sa))$, tức phần hậu tố còn lại, có thể xem $s_{T_N}(sa)$ như sa trong Trường hợp 2. Dù Trường hợp 2 dùng tính chất $sa = P_{T_w}$, bản thân $s_{T_N}(sa)$ đã có tính cực đại cần thiết, nên không gây sai lệch.

Bài toán còn lại là tìm $s_{T_N}(sa)$. Theo kết luận trên, ta cần hậu tố dài nhất của sa đã xuất hiện ít nhất một lần trong $F(T)$. Bắt đầu từ trạng thái $\Phi(T)_s$, đi lên cây parent qua các suffix link cho đến khi gặp trạng thái đầu tiên có chuyển nhãn a . Nếu trạng thái đó đại diện bởi một xâu r , thì ra là hậu tố dài nhất của sa xuất hiện ít nhất hai lần, tức $s_{T_N}(sa) = ra$.

Từ ba trường hợp trên, khi thêm một ký tự vào trie, số trạng thái tăng nhiều nhất hai. Điều này một lần nữa chứng minh số trạng thái tuyến tính, đồng thời mô tả được thay đổi của suffix link. Ta vẫn chưa xét cách sửa hàm chuyển; phần tiếp theo xử lý điểm này.

3.5.2 Thay đổi hàm chuyển

Trong Trường hợp 2, nếu không tạo trạng thái mới, chắc chắn không có thay đổi hàm chuyển. Nếu có, lớp cũ $R_T(sa)$ bị tách thành hai lớp $R_{T_N}(sa)$ và một lớp còn lại. Vì nút mới T_w là lá, nó không làm thay đổi các chuyển vốn có từ các xâu cũ. Các chuyển của hai trạng thái mới có thể sao chép từ trạng thái cũ tương ứng.

Tuy nhiên, vì lớp $R_T(sa)$ cũ bị xóa, có thể tồn tại các trạng thái có chuyển nhãn a trở tới trạng thái cũ đó; những chuyển này phải được sửa. Theo phân tích ở Trường hợp 2, các chuyển dọc theo chuỗi suffix link từ trạng thái của s lên trên, chừng nào chúng còn đi tới lớp cũ, sẽ được đổi để trở tới lớp mới phù hợp. Những chuyển khác giữ nguyên. Cách làm trực tiếp là giữ một bản sao đại diện cho phần không đổi, tạo một trạng thái mới cho phần bị tách, rồi bạo lực sửa các chuyển nhãn a trên chuỗi suffix link cho tới khi gặp trạng thái mà chuyển nhãn a không còn trở tới lớp cũ.

Trong Trường hợp 3, ta tạo trạng thái mới $R_{T_N}(sa)$; phần còn lại giống Trường hợp 2. Vì T_w là lá, trạng thái mới ban đầu không có chuyển ra. Do có trạng thái mới, một số trạng thái trên chuỗi suffix link có thể cần thêm chuyển nhãn a trở tới trạng thái mới của sa . Theo phân tích Trường hợp 3, chỉ cần thêm các chuyển này dọc theo chuỗi từ trạng thái của s đi lên cho tới khi gặp trạng thái đã có chuyển nhãn a thích hợp.

3.5.3 Phân tích độ phức tạp

Tương tự trường hợp một xâu, toàn bộ thuật toán là thuật toán tăng dần online. Với trie T , số lần thực hiện là $|T|$. Mỗi lần chỉ thêm nhiều nhất hai trạng thái, nên phần trạng thái là $O(|T|)$. Suffix link của mỗi trạng thái mới chỉ sửa một lần, phần này cũng là $O(|T|)$.

Xét hàm chuyển. Trong Trường hợp 2 có thể cần sao chép một tập chuyển; mỗi trạng thái có nhiều nhất $|A|$ chuyển, nên phần này là $O(|T| |A|)$. Phần phức tạp hơn là các lần bạo lực sửa chuyển khác. Có thể chứng minh tổng thời gian cho phần này không vượt quá tổng độ sâu các lá trong trie; gọi tổng đó là $G(T)$.

Ý tưởng chứng minh giống phân tích thế năng. Gọi $Dep(y)$, với $y \in F(T)$, là độ sâu của trạng thái $\Phi(T)_y$ trong cây parent. Xét đường từ trạng thái $\Phi(T)_s$ và trạng thái mới tới gốc trên cây parent. Theo Bổ đề 2.1(iii), nếu tồn tại lớp của sa thì cũng tồn tại lớp của s , nên $Dep(sa) \leq Dep(s) + 1$. Khi bạo lực sửa chuyển, ta liên tục đặt $c = s(c)$ và sửa chuyển của c ; mỗi bước như vậy làm $Dep(c)$ giảm 1. Khi dừng, độ sâu của trạng thái liên quan chỉ có thể tăng thêm một hằng số so với $Dep(s)$. Vì mỗi lần sửa đều làm giảm thế năng, tổng số lần sửa là cấp độ độ sâu.

Có thể xem Dep là hàm thế năng của trạng thái hiện tại. Với mỗi lá của T , nếu thực hiện riêng quá trình chèn theo đường từ gốc tới lá đó, mỗi phép chèn làm Dep tăng nhiều nhất 3, còn mỗi lần bạo lực sửa chuyển làm Dep giảm 1. Do đó số lần bạo lực sửa chuyển thực chất bị chặn bởi độ sâu; cộng trên mọi lá sẽ được kết luận trên.

Độ phức tạp này chưa phải đẹp, nhưng độ phức tạp của xây dựng online có thể đạt đến cấp đó.

Hình h. Nếu chèn các nút theo thứ tự chỉ số nút trong ví dụ của bản gốc, độ phức tạp đạt $O(G(T))$.

Khi không yêu cầu online, ta có thể thêm các nút trên trie theo thứ tự FIFO, tức thứ tự duyệt rộng. Khi đó chứng minh độ phức tạp của J. A. Blumer và cộng sự [9, 10] cho thuật toán xây trên xâu cũng áp

dụng cho trie. Chỉ những tổ hợp xâu con cố định mới khiến ta phải bạo lực sửa hàm chuyển [6], gợi ý rằng phần độ phức tạp này không vượt quá $O(|T|)$.

Tóm lại, nếu yêu cầu online, độ phức tạp xây dựng là $O(G(T))$. Nếu cho phép offline, ta có thể xây theo thứ tự FIFO và đạt độ phức tạp $O(|T||A|)$, đúng với cận dưới đã nêu.

3.6 Ứng dụng

3.6.1 ZJOI 2015: Vùng đất huyền tưởng được chú thần ưu ái

Đề bài đã xuất hiện ở phần lời nói đầu, nên không nhắc lại nữa.

Sau chuyển hóa, bài toán trở thành: đếm số xâu con khác nhau về bản chất của một trie. Bằng phương pháp xây suffix automaton cho trie ở trên, ta thu được một automaton có thể nhận ra mọi hậu tố của trie. Mỗi xâu con của trie chắc chắn là tiền tố của một hậu tố nào đó, giống hệt tính chất khi xây suffix automaton cho một xâu. Vì vậy, với một trạng thái $\Phi(T)_s$, số xâu con khác nhau mà nó đại diện là $|R(s)|$; cộng $|R(s)|$ trên mọi trạng thái sẽ cho đáp án.

Cũng có cách khác. Sau khi xây automaton, trực tiếp đếm có bao nhiêu xâu khác nhau không bị automaton từ chối; điều này tương đương đếm số xâu con khác nhau, hay đếm số đường chuyển khác nhau tồn tại khi bắt đầu từ trạng thái đầu. Xem automaton như một đồ thị có hướng không chu trình, ta chỉ cần quy hoạch động đơn giản trên đồ thị này.

Theo phân tích độ phức tạp ở trên, đề bài không yêu cầu online, nên có thể dùng thuật toán offline với độ phức tạp $O(n|A|)$. Thực ra điều kiện số lá là hằng số rất mạnh; có thể chứng minh automaton xây bằng phương pháp trên có thời gian $O(n)$. Lý do là ta có thể tách riêng đường từ gốc đến từng lá thành các xâu, rồi dùng phân tích độ phức tạp giống trường hợp xâu [9]. Vì điều này không liên quan trực tiếp đến nội dung chính của bài viết, tác giả bỏ qua chứng minh chi tiết.

Đến đây bài toán trong phần lời nói đầu đã được giải quyết trọn vẹn.

Từ bài toán này có thể thấy hầu hết bài toán xâu xử lý được bằng suffix automaton đều có thể mở rộng lên trie: tìm xâu con thứ K nhỏ nhất của trie, tìm xâu con chung dài nhất của nhiều trie, đếm số lần xuất hiện của xâu vòng trên trie, v.v. Cũng có những bài rất dễ trên xâu nhưng phức tạp hơn khi mở rộng lên trie, chẳng hạn đếm số xâu khác nhau mà tiền tố của chúng là một xâu cho trước. Các bài này đều có thể giải sau khi xây suffix automaton cho trie.

3.6.2 So sánh với AC automaton

Trong các thuật toán xâu quen thuộc, AC automaton xây một trie từ m mẫu rồi xây các con trỏ fail, sau đó có thể nhận diện số lần xuất hiện của từng mẫu trong xâu đầu vào. Ta có thể thay thế cách dùng này bằng suffix automaton trên trie: xây trie từ m xâu, rồi xây suffix automaton cho trie đó. Khi đọc xâu cần nhận diện, nếu trạng thái hiện tại $\Phi(T)_s$ không có chuyển nhãn a , ta nhảy tới $fa(\Phi(T)_s)$ và tiếp tục nhận diện. Rõ ràng tổng số lần nhảy không vượt quá độ dài xâu đang nhận diện.

Về độ phức tạp, trie T xây từ m xâu có $G(T)$ không vượt quá tổng độ dài của m xâu, nên độ phức tạp không kém hơn AC automaton. Nhưng thuật toán xây ở đây có thể là online, trong khi thuật toán xây AC automaton chỉ là offline.

3.6.3 Cây parent

Hình i. Bản gốc minh họa việc bổ sung các ký tự trên cạnh của Hình b để nhìn cây parent như một trie của các tiền tố đảo.

Thực chất, ta có thể đảo toàn bộ tiền tố trong trie rồi xây lại thành một trie, tương tự suffix tree của một xâu. Với mỗi trạng thái $\Phi(T)_s$, sắp xếp các con của nó trong cây parent. Cụ thể, nếu

$$Fa(\Phi(T)_x) = \Phi(T)_s,$$

ta sắp xếp các trạng thái $\Phi(T)_x$ theo thứ tự của đoạn

$$x_{1, |x| - \text{Maxl}(R(s))}.$$

Khi đó ta thu được một mảng tiền tố của trie, tức thứ tự sau khi đảo mọi tiền tố. Ta cần duy trì cho $\Phi(T)_s$ một nút $T_v \in \text{Right}(s)$; kết hợp nhân đôi trên cây, có thể nhanh chóng tìm được đoạn $x_{1, |x| - \text{Maxl}(R(s))}$.

Với tính chất này, có thể nhanh chóng xác định trạng thái tương ứng khi thêm một ký tự $a \in A$ vào trước xâu đang nhận diện. Giả sử hiện ở trạng thái $\Phi(T)_s$. Ta tìm nhị phân trong các con của nó trên cây parent để tìm $\Phi(T)_{as}$, hoặc kết luận $\Phi(T)_{as} = \Phi(T)_s$.

3.7 Lời cảm ơn

Xin cảm ơn thầy Qu Yunhua của Trường Yali đã lâu dài hướng dẫn và quan tâm tới việc học tập cũng như cuộc sống của tôi.

Xin cảm ơn cha mẹ đã nuôi dưỡng tôi.

Xin cảm ơn các thầy Zhu Quanmin và Wang Xingming của Trường Yali đã chỉ dạy.

Xin cảm ơn CCF đã cung cấp nền tảng và cơ hội giao lưu.

Xin cảm ơn các huấn luyện viên đội tuyển quốc gia vì sự tận tụy.

Xin cảm ơn các anh Huang Zhixiang của Đại học Thanh Hoa và Huang Yuyang của Đại học Giao thông Thượng Hải đã giúp đỡ tôi.

Xin cảm ơn các bạn Yang Dingcheng, Kuang Zhengfei và Liu Jiancheng của Trường Yali đã đọc duyệt bài viết này.

Xin cảm ơn tất cả những bạn học từng giúp đỡ tôi.

Tài liệu tham khảo của bài viết

- [1] A. V. Aho and M. J. Corasick, Efficient string matching: An aid to bibliographic research.
- [2] R.-S. Boyer and J. S. Moore, A fast string searching algorithm.
- [3] D. E. Knuth, J. H. Morris and V. I. Pratt, Fast pattern-matching in strings.
- [4] P. Weiner, Linear pattern-matching algorithms.
- [5] E. M. McCreight, A space-economical suffix-tree construction algorithm.
- [6] M. Mohri, P. Moreno and E. Weinstein, General Suffix Automaton Construction Algorithm and Space Bounds.
- [7] M. Crochemore, Transducers and repetitions.
- [8] Chen Lijie, *Suffix automaton*.
- [9] J. A. Blumer, Algorithms for the directed acyclic word graph and related structures.
- [10] A. Blumer, J. Blumer, D. Haussler, R. M. McConnell and A. Ehrenfeucht, Complete inverted files for efficient text retrieval and analysis.
- [11] U. Manber and G. Myers, Suffix arrays: A new method for on-line string searches.

[12] Luo Suijian, *Suffix array: một công cụ mạnh để xử lý xâu*.

[13] Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest and Clifford Stein, *Introduction to Algorithms*.

4 Bàn về ứng dụng tư tưởng heuristic trong thi tin học

Ren Zhizhou

Trường Trung học số 1 Thiệu Hưng

Tóm tắt

Thuật toán heuristic là các thuật toán được thiết kế dựa trên kinh nghiệm. So với các thuật toán tối ưu hóa truyền thống, heuristic thường đánh đổi độ chính xác hoặc tính ổn định để lấy tốc độ. Bài viết thử đưa ra một định nghĩa cụ thể hơn cho heuristic trong thi tin học, đồng thời thảo luận một số thuật toán heuristic thường dùng và một số mô hình bài toán. Trong đó có một thuật toán xấp xỉ cho cây Steiner nhỏ nhất với độ phức tạp $O(|S||V|^2)$; bài viết cũng chứng minh tỉ lệ xấp xỉ của thuật toán này đạt

$$2 \left(1 - \frac{1}{\text{leaf}} \right) \leq 2 \left(1 - \frac{1}{|S|} \right),$$

trong đó leaf là số lá của cây Steiner tối ưu.

4.1 Dẫn nhập

Thuật toán heuristic là một lớp thuật toán được nêu ra trong tương quan với các thuật toán tối ưu hóa. Chúng được xây dựng từ trực giác hoặc kinh nghiệm, với mục tiêu tìm được một nghiệm khá tốt trong chi phí thời gian và bộ nhớ chấp nhận được. Khái niệm này rất rộng; đa số heuristic khá phức tạp và không thể đưa thẳng vào bài thi OI.

Tác giả giản lược và cải tiến một số thuật toán heuristic để chúng phù hợp hơn với OI, rồi định nghĩa thêm hai kiểu heuristic trong bối cảnh thi tin học:

- dựa vào trực giác để điều chỉnh và tối ưu thuật toán, hy sinh tính ổn định để đổi lấy tốc độ;
- dựa vào trực giác để trực tiếp dựng nghiệm tốt, hy sinh tính đúng tuyệt đối để giải nhanh.

Nhiều thuật toán quen thuộc cũng chứa tư tưởng heuristic. Sau tối ưu, chúng không làm giảm độ phức tạp xấu nhất, nhưng hiệu suất thực tế thường tăng rõ rệt, chẳng hạn:

- truy vấn điểm gần nhất trên k -d tree;
- Bellman–Ford tối ưu bằng hàng đợi, tức SPFA, và các tối ưu tham lam liên quan;
- mô phỏng tôi luyện, thuật toán di truyền và các thuật toán tìm kiếm ngẫu nhiên lấy cảm hứng từ khoa học tự nhiên.

Bài viết thảo luận ba mô hình heuristic:

- thuật toán A^* và các biến thể, dùng để nối giữa thuật toán xấp xỉ và lời giải tối ưu;
- phân loại Bayes ngây thơ, một thuật toán xác suất kết hợp phán đoán trực quan với suy luận toán học;
- gộp heuristic, một tối ưu đơn giản cho các bài toán cần hợp nhất dữ liệu.

Ngoài việc áp dụng mô hình kinh điển, nhiều khi ta cũng có thể tự thiết kế heuristic. Bài viết dành nhiều dung lượng cho hai bài toán cổ điển:

- bài toán chia đều có trọng số, hay *2-partition problem*, từ đó có một thuật toán xấp xỉ có xác suất đúng rất cao và quy trình thiết kế đáng tham khảo;
- bài toán cây Steiner, được trình bày kỹ hơn, với một thuật toán hiệu quả có thể đánh giá sai số xấp xỉ, rồi kết hợp với A^* để thu được thuật toán xấp xỉ cho K cây Steiner nhỏ nhất.

Ở mỗi phần, tác giả đều đưa ví dụ với hy vọng giúp người đọc nắm được cách sử dụng và tiếp tục phát triển các ý tưởng này.

4.2 Thuật toán A^*

Trong một số bài toán rất khó tìm thuật toán đa thức hiệu quả, ta buộc phải dùng tìm kiếm, trong khi số trạng thái lại rất lớn. Khi đó có thể thiết kế hàm ước lượng để cắt tỉa; phần này trình bày một số cách dùng ước lượng để tăng tốc tìm kiếm.

4.2.1 Tìm kiếm heuristic

Giả sử trạng thái hiện tại là s , chi phí thực tế đã đi tới s là $g(s)$, ước lượng chi phí từ s tới trạng thái đích là $h(s)$, còn chi phí tối ưu thật sự từ s tới đích là $h^*(s)$. Đặt

$$F(s) = g(s) + h(s).$$

Ta có thể dựa vào giá trị này, mỗi lần mở rộng trạng thái có $F(s)$ tốt nhất, và cắt tỉa theo tính chất của hàm ước lượng. Trong bài viết, các bài toán được mặc định là tối thiểu hóa chi phí.

4.2.2 $h(s) = h^*(s)$

Khi hàm ước lượng cho đúng chi phí còn lại, mỗi lần mở rộng trạng thái tốt nhất theo $F(s)$ sẽ nhanh chóng thu được nghiệm tối ưu.

Sau khi đã có nghiệm tối ưu, nếu tiếp tục mở rộng các trạng thái còn lại thì có thể nhận được nghiệm tốt thứ K , đồng thời tránh tính các trạng thái thừa. Phần tổng quát và các tối ưu cho bài toán đặc biệt đã được Yu Yili trình bày trong luận văn đội tuyển quốc gia năm 2014.

4.2.3 $h(s) \leq h^*(s)$

Đôi khi, để tính ước lượng thuận tiện, ta bỏ qua một số ràng buộc trong đề bài. Kết quả thu được là một nghiệm thô “tốt hơn” nghiệm tối ưu thật, tức ước lượng không vượt quá chi phí còn lại. Khi đó ước lượng vẫn cung cấp căn cứ cắt tỉa đúng, giúp giảm phạm vi trạng thái phải xét.

IDA*. IDA*, hay tìm kiếm sâu dần lặp có heuristic, là một biến thể của A^* kết hợp ý tưởng lặp sâu dần với hàm ước lượng. Trong các bài có số trạng thái lớn và khó lưu trữ, cách này tiết kiệm rất nhiều bộ nhớ, đồng thời mã cài đặt cũng gọn.

Với lặp sâu dần đơn giản, mỗi lượt thường chọn một bước tăng cố định và dùng giới hạn độ sâu để cắt tỉa. Với IDA*, ta cắt tỉa bằng ước lượng. Vì $h(s) \leq h^*(s)$, giá trị nhỏ nhất của $g(s) + h(s)$ trong các trạng thái bị cắt có thể dùng để ước lượng đáp án kế tiếp, giảm số lần lặp so với bước tăng cố định.

Ví dụ 1: UVa 10181, 15-Puzzle Problem. Bài toán 15-puzzle mở rộng từ 8-puzzle. Trên bảng 4×4 có 15 ô đánh số và một ô trống; mỗi bước có thể đưa một ô kề ô trống vào vị trí trống. Cần tìm số bước ít nhất để tới trạng thái đích. Dữ liệu bảo đảm mọi trường hợp có lời giải đều tìm được trong không quá 45 bước.

Việc phán đoán vô nghiệm cần phân tích riêng về puzzle số, bài viết không đi sâu. Với phần tìm kiếm, một ước lượng rất đơn giản là tổng khoảng cách Manhattan từ mỗi ô không trống tới vị trí đích. Cách này tương đương với việc bỏ qua ràng buộc “chỉ được đổi chỗ với ô trống”, đồng thời bỏ qua ảnh hưởng của một phép đổi chỗ tới ô còn lại.

Vì đã bỏ bớt ràng buộc, ta có $h(s) \leq h^*(s)$. Với dữ liệu có lời giải trong 45 bước, hiệu quả cắt tỉa và hiệu quả giảm số vòng lặp đều khá tốt.

4.2.4 Anytime algorithm

Lớp này còn được gọi là thuật toán có thể ngắt giữa chừng. Không phải bài nào cũng dễ tìm hàm ước lượng hiệu quả. Đôi khi ta chỉ ước lượng rất thô, không kiểm soát được $h(s) \leq h^*(s)$, nên cũng không thể thu hẹp không gian tìm kiếm một cách đúng đắn. Những bài như vậy thường xuất hiện dưới dạng nộp đáp án. Ta có thể dùng thuật toán cải thiện dần, trong thời gian thi cố gắng thu được nghiệm càng tốt càng tốt.

Ví dụ 2: CTSC 2009 Day 2, trò chơi số $N \times N$. Trò chơi gồm $N^2 - 1$ ô trượt và một ô trống. Mục tiêu là di chuyển ô trống lên, xuống, trái, phải để đưa các ô về một thứ tự quy định. Đây là bài nộp đáp án, với $3 \leq N \leq 6$ và 10 bộ dữ liệu.

So với ví dụ 1, miền dữ liệu lớn hơn nhiều. Đổi lại, bài không yêu cầu nghiệm chính xác; chỉ cần trong thời gian cho phép tìm được nghiệm đủ tốt thì vẫn có điểm đáng kể.

Nếu dùng trực tiếp ước lượng Manhattan của ví dụ 1 rồi chạy IDA*, ta có thể nhanh chóng tìm được nghiệm tối ưu cho 5 bộ nhỏ. Nhưng giá trị ước lượng vẫn cách đáp án thật khá xa, nên các bộ còn lại không xử lý được.

Một cách sửa đơn giản là đặt

$$h'(s) = c h(s),$$

với c là hằng số điều chỉnh theo từng bộ dữ liệu. So với $h(s)$, $h'(s)$ gần số bước thật hơn, nên có thể đưa vào A^* . Tuy vậy, A^* tốn bộ nhớ rất lớn, đó cũng là lý do ví dụ 1 không dùng A^* .

Với bài nộp đáp án, ta có thể thỏa hiệp: chỉ giữ lại một phần trạng thái có ước lượng tốt nhất. Mỗi lần chọn trạng thái tốt nhất để mở rộng; nếu không còn chỗ lưu trạng thái mới thì bỏ trạng thái kém nhất. Vì bài có rất nhiều phương án lời giải, việc loại bỏ một số trạng thái “xấu” ít ảnh hưởng đến đáp án cuối. Ngoại trừ một bộ đặc biệt, cách này có thể đạt khoảng 9 đến 10 điểm mỗi bộ.

4.3 Gộp heuristic

Gộp heuristic là một chiến lược hợp nhất đơn giản, có thể tối ưu hiệu quả cho các thuật toán cần gộp dữ liệu.

4.3.1 Gộp heuristic trên cấu trúc dữ liệu

Giả sử cần gộp hai cấu trúc dữ liệu A, B , số phần tử đang được chúng quản lý lần lượt là a, b và $a \leq b$. Ta tách toàn bộ thông tin trong cấu trúc nhỏ A , rồi lần lượt chèn chúng vào cấu trúc lớn B .

Định lý 3.1. Giả sử sau một số phép gộp ta thu được một cấu trúc dữ liệu có n phần tử. Khi dùng chiến lược gộp heuristic, tổng số lần chèn phát sinh là $O(n \log n)$.

Chứng minh. Xét riêng từng phần tử. Khi một phần tử bị chèn sang cấu trúc khác, kích thước cấu trúc chứa nó sau phép gộp ít nhất gấp đôi trước đó. Sau $O(\log n)$ lần chèn, phần tử đó chắc chắn đã nằm trong cấu trúc cuối cùng. Cộng trên mọi phần tử, ta được $O(n \log n)$.

Chiến lược này có thể gộp các cấu trúc dữ liệu phức tạp, và cũng dễ mở rộng cho các kiểu tập dữ liệu khác.

Ví dụ 3: HNOI 2009, Bánh pudding huyền ảo. Có n chiếc pudding xếp thành một hàng và m thao tác. Mỗi lần đổi toàn bộ pudding của một màu thành màu khác, rồi hỏi hiện có bao nhiêu đoạn màu liên tiếp. Dữ liệu có $n \leq 10^6$.

Ta dùng danh sách liên kết hoặc vector để lưu các vị trí của từng màu. Khi đổi màu, chọn danh sách nhỏ hơn để tách thô, sửa từng vị trí và cập nhật đóng góp vào số đoạn. Theo Định lý 3.1, tổng số lần sửa là $O(n \log n)$, nên có thể bảo trì trực tiếp.

Ví dụ 4: BZOJ 3510, Thủ đô. Cần duy trì một rừng n đỉnh với ba thao tác:

- nối hai đỉnh u, v , bảo đảm sau khi nối vẫn là rừng;
- hỏi trọng tâm của cây chứa đỉnh x , nếu có hai trọng tâm thì lấy chỉ số nhỏ hơn;
- hỏi xor chỉ số trọng tâm của tất cả các cây.

Trước hết xét việc thêm một lá vào một cây. Trọng tâm chỉ có thể di chuyển về phía lá mới, và nhiều nhất di chuyển một lần.

Khi nối hai cây, ta tách thô cây có ít đỉnh hơn và thêm từng đỉnh vào cây lớn dưới dạng lá, đồng thời duy trì trọng tâm. Theo Định lý 3.1, số lần thêm lá là $O(n \log n)$; chỉ cần thêm một cấu trúc dữ liệu để kiểm tra trọng tâm có di chuyển hay không.

4.3.2 Hợp nhất theo hạng trong DSU

Hợp nhất theo hạng là một cách duy trì DSU rất thực dụng: mỗi lần nối gốc của tập nhỏ vào gốc của tập lớn. Từ Định lý 3.1 suy ra chiều cao cây là $O(\log n)$. Khi cài đặt, cũng có thể trực tiếp duy trì độ cao rồi hợp nhất theo hạng.

Cách này bảo đảm độ sâu cây DSU ngay cả khi không nén đường đi. Trong một số bài, nó tránh được việc mất thông tin khi nén đường đi.

Ví dụ 5: NOIP 2013, Vận tải bằng xe tải. Có n thành phố, m đường hai chiều; mỗi đường có giới hạn trọng lượng z_i . Với q xe tải, cần biết mỗi xe có thể chở tối đa bao nhiêu mà không vượt quá giới hạn trên đường đi. Dữ liệu:

$$n \leq 10000, \quad m \leq 50000, \quad q \leq 30000, \quad z_i \leq 100000.$$

Cách thông thường là dùng Kruskal dựng cây khung lớn nhất. Giá trị nhỏ nhất trên đường đi trong cây khung lớn nhất chính là giới hạn lớn nhất có thể giữa hai đỉnh; có thể trả lời bằng nhân đôi, với độ phức tạp $O(m \log m + q \log n)$.

Nếu dùng hợp nhất theo hạng thay cho nén đường đi, ta có thể ghi lại với mỗi đỉnh trong DSU trọng số cạnh nối nó với cha. Giá trị nhỏ nhất trên đường trong cây DSU vẫn biểu diễn giới hạn rộng nhất giữa

hai đỉnh. Trả lời thô có độ phức tạp $O(m \log m + q \log n)$, nhưng cài đặt ngắn hơn cách dựng cây rồi nhân đôi.

Vì bài toán chuyển hóa vẫn là truy vấn giá trị nhỏ nhất trên đường đi, ta vẫn có thể dùng nhân đôi; về lý thuyết phần hồi đạt $O(\log \log n)$. Nếu chọn thuật toán sắp xếp tuyến tính và vẫn nén đường đi trong DSU theo cách nối cạnh ban đầu, độ phức tạp lý thuyết có thể đạt

$$O(n + m\alpha(n) + q \log \log n).$$

4.4 Bài toán trừu tượng và học máy

Khi gặp các bài toán phân loại trừu tượng, cảm giác trực quan thường không cho căn cứ phán đoán chính xác. Khi đó có thể dùng học máy thay cho quan sát thủ công. Phần này thông qua một bài Google Code Jam để giới thiệu ngắn gọn một thuật toán phân loại thực dụng.

4.4.1 Bayes ngây thơ

Công thức Bayes. Ký hiệu $P(A | B)$ là xác suất sự kiện A xảy ra khi đã biết B xảy ra; ký hiệu $P(A \cap B)$ là xác suất cả A và B cùng xảy ra.

Định lý 4.1.

$$P(A | B) = \frac{P(B | A)P(A)}{P(B)}.$$

Chứng minh. Ta có

$$P(A \cap B) = P(A)P(B | A) = P(B)P(A | B).$$

Chuyển về sẽ thu được công thức Bayes.

Phán đoán ngây thơ. Gọi S là sự kiện cần phân loại, A và B là hai lớp có thể chọn. Nếu $P(A | S) > P(B | S)$, ta cho rằng S thuộc lớp A ; ngược lại thuộc lớp B . Khi có nhiều lớp hơn, chọn lớp có xác suất lớn nhất.

Giả định ngây thơ. Giả sử sự kiện cần phân loại có các thuộc tính

$$S = \{S_1, S_2, \dots, S_m\}.$$

Nếu giả định các thuộc tính đặc trưng này độc lập với nhau khi đã biết lớp, ta có thể xấp xỉ

$$P(S | A) \approx \prod_{i=1}^m P(S_i | A).$$

So sánh các xác suất xấp xỉ như vậy sẽ cho kết quả phân loại. Các giá trị $P(S_i | A)$ có thể tính bằng lấy mẫu ngẫu nhiên hoặc bằng một thuật toán riêng cho bài toán.

4.4.2 Ví dụ 6: GCJ Round 1A 2014, *Proper Shuffle*

Có hai cách sinh hoán vị. Đề cho 120 hoán vị độ dài 1000, mỗi hoán vị được sinh độc lập bởi một trong hai thuật toán. Cần nhận diện hoán vị nào do thuật toán nào sinh ra; nếu trả lời đúng ít nhất 109 trong 120 hoán vị thì được chấp nhận.


```

Algorithm 1 GOOD
for k = 0 to n - 1 do
    a[k] = k
end for
for k = 0 to n - 1 do
    p = random_int(k..n - 1)
    swap(a[k], a[p])
end for

```

```

Algorithm 2 BAD
for k = 0 to n - 1 do
    a[k] = k
end for
for k = 0 to n - 1 do
    p = random_int(0..n - 1)
    swap(a[k], a[p])
end for

```

Thuật toán GOOD sinh mọi hoán vị với xác suất bằng nhau, còn BAD thì không.

Phân tích đơn giản. Với GOOD, sau khi quyết định vị trí k , xác suất xuất hiện của mọi giá trị tại vị trí đó đều là $1/n$, và các bước sau không ảnh hưởng vị trí đó.

Với BAD, khi quyết định phép đổi tại vị trí k , nếu $\text{swap}(a[k], a[p])$ có $p < k$, giá trị vốn ở vị trí $a[k]$ bị đổi tới một vị trí cao hơn, rồi sau đó còn có thể tiếp tục bị đổi tới vị trí cao hơn nữa. Điều này tạo ảnh hưởng tinh vi tới xác suất một giá trị xuất hiện tại từng vị trí.

Một đặc trưng đơn giản. Gọi S là hoán vị đầu vào. Có thể dùng số lượng vị trí thỏa $S_i < i$ làm đặc trưng. Phân tích của cuộc thi cho thấy phân bố đặc trưng này giữa GOOD và BAD khác nhau rõ rệt; nếu dùng một ngưỡng thích hợp thì tỉ lệ đúng có thể vượt 90%.

Phân loại Bayes ngây thơ. Ta muốn tính $P(\text{GOOD} \mid S)$. Theo Định lý 4.1:

$$P(\text{GOOD} \mid S) = \frac{P(S \mid \text{GOOD})P(\text{GOOD})}{P(S \mid \text{GOOD})P(\text{GOOD}) + P(S \mid \text{BAD})P(\text{BAD})}.$$

Vì $P(\text{GOOD}) = P(\text{BAD})$, công thức rút gọn thành

$$P(\text{GOOD} \mid S) = \frac{P(S \mid \text{GOOD})}{P(S \mid \text{GOOD}) + P(S \mid \text{BAD})},$$

và tương tự

$$P(\text{BAD} \mid S) = \frac{P(S \mid \text{BAD})}{P(S \mid \text{GOOD}) + P(S \mid \text{BAD})}.$$

Nếu $P(\text{GOOD} \mid S) > P(\text{BAD} \mid S)$, xác suất S do GOOD sinh ra lớn hơn. Điều kiện này tương đương

$$P(S \mid \text{GOOD}) > P(S \mid \text{BAD}).$$

Với GOOD, mọi hoán vị cùng xác suất:

$$P(S \mid \text{GOOD}) = \frac{1}{n!}.$$

Việc tính chính xác $P(S \mid \text{BAD})$ lại rất khó, nên ta xấp xỉ:

$$P(S \mid \text{BAD}) \approx \prod_{k=0}^{n-1} P(S_k \mid \text{BAD}).$$

Để so sánh công bằng hơn, ta cũng dùng xấp xỉ tương tự cho GOOD:

$$P(S \mid \text{GOOD}) \approx \prod_{k=0}^{n-1} \frac{1}{n}.$$

Nếu $P_{i,j}$ là xác suất sau khi chạy BAD ta có $S_i = j$, ta dễ thiết kế một DP $O(n^2)$ để tính toàn bộ bảng này. Từ đó có thể tính $P(S \mid \text{BAD})$ xấp xỉ rồi so sánh với $P(S \mid \text{GOOD})$. Khi tính trực tiếp xác suất có thể gặp yêu cầu độ chính xác cao; thực tế có thể so sánh $P(S \mid \text{BAD}) n^n$ với 1.0, hoặc dùng ngưỡng khác vì bản thân quá trình đã là xấp xỉ.

4.5 Bài toán chia đều có trọng số

Ví dụ 7. Cho một đa tập số nguyên dương S . Ký hiệu $f(S)$ là tổng các phần tử trong S . Chia S thành hai tập S_1, S_2 , với $S_1 + S_2 = S$, sao cho $|f(S_1) - f(S_2)|$ nhỏ nhất, đồng thời đưa ra phương án chia.

4.5.1 Ví dụ đơn giản

Với $S = \{3, 1, 1, 2, 2, 1\}$, có hai cách chia hợp lệ làm $f(S_1) = f(S_2) = 5$, tức $|f(S_1) - f(S_2)| = 0$:

$$S_1 = \{1, 1, 1, 2\}, \quad S_2 = \{2, 3\};$$

$$S_1 = \{3, 1, 1\}, \quad S_2 = \{2, 2, 1\}.$$

4.5.2 Một thuật toán quy hoạch động

Có thể thiết kế DP giống ba lô: đặt $d[i][j]$ cho biết sau khi dùng i phần tử đầu tiên, có thể đạt $f(S_1) = j$ hay không. Độ phức tạp là $O(nf(S))$.

Khi mọi phần tử trong S đều nhỏ, đây đúng là thuật toán tốt. Nhưng thực chất đây chỉ là thuật toán giả đa thức.

4.5.3 Một ý tưởng tham lam đơn giản

Ý tưởng tự nhiên là đọc lần lượt từng phần tử và đặt nó vào tập hiện có tổng nhỏ hơn. Để hai tổng dần áp sát nhau, ta sắp xếp các phần tử giảm dần trước khi xử lý; độ phức tạp là $O(n \log n)$.

Thuật toán này rất tự nhiên và kết quả thường không tệ, nhưng phân tích kỹ sẽ thấy nó có lỗ hổng rõ ràng.

4.5.4 Thuật toán sai phân

Thuật toán sai phân cải tiến thuật toán tham lam ở trên. Ý tưởng ngắn gọn, nhưng chất lượng nghiệm xấp xỉ tốt hơn nhiều.

Tư tưởng cơ bản. Khi đặt hai số X, Y vào hai tập khác nhau, phần ảnh hưởng tới đáp án chỉ là độ lớn của hiệu giữa chúng. Điều này tương đương tạo ra một số mới $|X - Y|$, rồi tiếp tục xem nó như một số thường trong thuật toán.

Nhìn lại thuật toán tham lam trước đó, xét trạng thái ở mỗi quyết định. Giả sử phần tử mới là X , còn các phần tử đã quyết định tạo ra sai phân Y . Việc đưa X vào tập có tổng nhỏ hơn tương đương thay X, Y bằng $|X - Y|$. Nhưng mục đích ban đầu của ta là xử lý phần tử lớn trước, nên quyết định theo thứ tự đọc vào là chưa tối ưu.

Quy trình thuật toán. Theo tư tưởng sai phân, ta dùng heap và DSU để bảo trì:

1. Đưa mọi phần tử của S vào heap. Trong DSU, mỗi phần tử ban đầu là một tập riêng.
2. Lấy hai phần tử lớn nhất khỏi heap, gọi là X, Y ; trong DSU chúng tương ứng với hai tập S_X, S_Y . Quyết định đặt S_X và S_Y ở hai phía khác nhau, đưa $|X - Y|$ trở lại heap, và nối hai tập trong DSU bằng một cạnh trọng số 1 biểu diễn việc đổi phía.
3. Nếu heap chỉ còn một phần tử thì dừng, ngược lại lặp bước trước.
4. Chọn gốc của cây DSU thuộc S_1 . Các phần tử khác dựa vào tính chẵn lẻ của độ dài đường tới gốc để quyết định thuộc S_1 hay S_2 : chẵn cùng phía, lẻ khác phía.

Khi cài đặt DSU, có thể duy trì tính chẵn lẻ của độ dài đường trong lúc nén đường đi, hoặc dùng hợp nhất theo hạng để bảo đảm chiều cao $O(\log n)$.

Hiệu quả xấp xỉ. Thuật toán có độ phức tạp $O(n \log n)$, rất hiệu quả; điều quan trọng là nghiệm có đủ tốt hay không.

Khi n không quá nhỏ, chẳng hạn $n > 1000$, với tập số sinh ngẫu nhiên $S_i \in [0, 2^{30})$, thuật toán gần như chắc chắn chia được S_1, S_2 sao cho

$$|f(S_1) - f(S_2)| \leq 1.$$

Nói cách khác, nếu $f(S)$ lẻ thì sai khác bằng 1, còn nếu $f(S)$ chẵn thì sai khác bằng 0. Có thể thấy xác suất đúng của thuật toán này rất cao.

Khi n nhỏ, có thể dùng thuật toán này để thiết kế hàm ước lượng rồi chạy tìm kiếm heuristic nhằm thu được nghiệm tốt hơn.

4.5.5 Ví dụ 8: POI 2013, *Polarization*

Cho một cây n đỉnh. Hãy định hướng mỗi cạnh thành cạnh có hướng. Nếu từ đỉnh u đi được tới đỉnh v , gọi (u, v) là một cặp hợp lệ. Cần tìm số cặp hợp lệ nhỏ nhất và lớn nhất trong mọi cách định hướng. Dữ liệu có $n \leq 250000$.

Chuyển hóa bài toán. Cây là đồ thị hai phía, nên giá trị nhỏ nhất chắc chắn là $n - 1$.

Với giá trị lớn nhất, nghiệm tối ưu luôn tồn tại một đỉnh x sao cho mọi đỉnh còn lại hoặc có đường đi tới x , hoặc có đường đi từ x tới nó. Phân tích tiếp cho thấy x phải chọn ở trọng tâm của cây.

Lấy x làm gốc, cây tách thành một số cây con. Bài toán còn lại là chia các cây con này thành hai tập sao cho tích kích thước của hai tập lớn nhất.

Thuật toán tối ưu truyền thống. Chú ý rằng tổng các kích thước ở đây là n , nên số giá trị phần tử khác nhau chỉ là $O(\sqrt{n})$. Ta có thể chuyển thành bài toán ba lô đa trọng, một bài toán kinh điển không trình bày lại ở đây. Độ phức tạp là $O(n\sqrt{n})$.

Thuật toán xấp xỉ. Nếu dùng trực tiếp thuật toán sai phân, có thể qua khoảng 80% dữ liệu. Nguyên nhân chính là khi số phần tử ít, sai lệch của nghiệm xấp xỉ lớn hơn.

Ta đặt một ngưỡng. Khi số cây con nhỏ hơn ngưỡng này, dùng ba lô vét cạn; các trường hợp còn lại dùng thuật toán sai phân. Cách ghép này có thể qua toàn bộ dữ liệu chính thức của POI, với độ phức tạp $O(n \log n)$.

Khi thuật toán tối ưu truyền thống đủ nhanh, không nên ưu tiên thuật toán xấp xỉ. Ví dụ này chỉ nhằm cho thấy thuật toán sai phân có thể tạo ra nghiệm xấp xỉ rất tốt.

4.6 Bài toán cây Steiner nhỏ nhất

Ví dụ 9. Cho đồ thị vô hướng có trọng số $G = (V, E)$. Tập đỉnh đặc biệt S là một tập con của V . Cần tìm một đồ thị liên thông

$$T = (V_T, E_T)$$

sao cho $S \subseteq V_T \subseteq V$, $E_T \subseteq E$, $|E_T| = |V_T| - 1$, và tổng trọng số các cạnh trong E_T là nhỏ nhất.

4.6.1 Một thuật toán tối ưu

Cây Steiner nhỏ nhất có thuật toán trạng thái nén kinh điển, chuyển về mô hình đường đi ngắn nhất; có thể xem luận văn đội tuyển quốc gia năm 2014 của Jiang Jinye. Độ phức tạp là

$$O(|V| 3^{|S|} + |E| 2^{|S|}),$$

đây là thuật toán tốt nhưng chỉ áp dụng được cho miền dữ liệu nhỏ.

4.6.2 Một ý tưởng tham lam đơn giản

Một tham lam đơn giản là tìm cây khung nhỏ nhất của đồ thị gốc, rồi xóa các cạnh thừa trên cây khung, tức xóa nhiều nhất có thể nhưng vẫn giữ cho tập S liên thông.

Thuật toán này có lỗi hổng lớn. Trong ví dụ của bản gốc, với

$$V = \{A, B, C, 1, 2, 3, 4, 5\}, \quad S = \{A, B, C\}, \quad x > 2,$$

tham lam theo cây khung nhỏ nhất chọn các cạnh trọng số x và cho nghiệm kém, trong khi nghiệm tối ưu đi qua các cạnh trọng số 1 và các cạnh $x + 1$ như hình minh họa.

4.6.3 Thuật toán heuristic

Dựa trên ví dụ trên, ta cải tiến có chủ đích.

Phân tích thêm. Trong ví dụ, nếu tạm bỏ qua các điểm 1 và 2, rồi xét khoảng cách ngắn nhất giữa các điểm còn lại, cây khung nhỏ nhất trên đồ thị rút gọn đã có thể cho nghiệm tối ưu. Để tổng quát hơn, ta loại cả những điểm không thuộc S khỏi đồ thị rút gọn.

Vấn đề là các đường đi ngắn nhất giữa các cặp đỉnh của S có thể giao nhau. Khi cuối cùng tạo cây, các cạnh giao nhau không được tính lặp.

Quy trình thuật toán. Sau khi chỉnh lý, ta có thuật toán sau:

1. Từ đồ thị gốc $G = (V, E)$, dựng đồ thị đầy đủ $G_1 = (S, E_1)$. Trọng số cạnh giữa hai đỉnh trong S là độ dài đường đi ngắn nhất giữa chúng trong G .
2. Tìm cây khung nhỏ nhất T_1 của G_1 . Nếu có nhiều nghiệm thì chọn tùy ý.
3. Dựng một đồ thị con $G_2 = (V_2, E_2)$ của G : với mỗi cạnh của T_1 , đưa vào G_2 một đường đi ngắn nhất tương ứng trong G . Nếu có nhiều đường đi ngắn nhất thì chọn tùy ý.
4. Tìm cây khung nhỏ nhất T_2 của G_2 . Nếu có nhiều nghiệm thì chọn tùy ý.
5. Xóa khỏi T_2 các cạnh vô dụng sao cho mọi lá còn lại đều là đỉnh thuộc S . Kết quả là cây Steiner T .

Phân tích từng bước:

- Dựng G_1 cần chạy đường đi ngắn nhất một nguồn $|S|$ lần, độ phức tạp $O(|S||V|^2)$ nếu dùng cách cơ bản.
- Tìm cây khung nhỏ nhất của G_1 ; vì G_1 là đồ thị đầy đủ, nên dùng Prim với độ phức tạp $O(|S|^2)$.
- Khi dựng G_2 , mỗi đường đi ngắn nhất có thể qua $O(|V|)$ đỉnh, nhưng tổng số cạnh đường đi được thêm vẫn bị chặn bởi $O(|V|)$ theo phân tích của bản gốc cho trường hợp này.
- Tìm cây khung nhỏ nhất của G_2 , độ phức tạp $O(|V|^2)$ hoặc $O(|V| \log |V|)$ tùy cài đặt.
- Xóa cạnh vô dụng trong $O(|V|)$.

Có thể thấy thuật toán chạy rất hiệu quả, đồng thời chất lượng xấp xỉ cũng tốt. Phần tiếp theo chúng mình chặn trên tương đối cho nghiệm xấp xỉ của nó. Có thể thay bước đường đi ngắn nhất bằng thuật toán hiệu quả hơn để tối ưu thêm độ phức tạp.

Hiệu quả xấp xỉ. Gọi D_T là tổng trọng số cây Steiner T do thuật toán trên thu được, và $D_{\min}$ là tổng trọng số cây Steiner nhỏ nhất $T_{\min}$. Ta sẽ chặn tỉ số $D_T/D_{\min}$.

Bổ đề 6.1. Với một cây T có $m > 1$ cạnh, tồn tại một vòng

$$u_0, u_1, \dots, u_{2m} = u_0$$

thỏa các tính chất:

- mỗi cạnh của T xuất hiện đúng hai lần trong vòng;
- mỗi lá của T chỉ xuất hiện một lần trong vòng, không tính việc u_0 lặp lại ở cuối;
- nếu u_i và u_j , với $i < j$, là hai lá liên tiếp trên vòng, và giữa chúng không có lá nào khác, thì $u_i, u_{i+1}, \dots, u_j$ là một đường đi đơn trên T .

Chứng minh. Chọn một đỉnh làm gốc, rồi duyệt Euler trên cây T . Dãy Euler thu được thỏa các tính chất trên nếu xem mỗi cạnh vô hướng như hai cạnh có hướng ngược nhau.

Định lý 6.1. Gọi leaf là số lá của cây Steiner nhỏ nhất $T_{\min}$. Khi đó

$$\frac{D_T}{D_{\min}} < 2 \left(1 - \frac{1}{\text{leaf}} \right) \leq 2 \left(1 - \frac{1}{|S|} \right).$$

Chứng minh. Vì $\text{leaf} \leq |S|$, bất đẳng thức bên phải là hiển nhiên.

Theo Bổ đề 6.1, trong $T_{\min}$ có thể tạo một vòng L bằng cách đi mỗi cạnh hai lần. Nếu trọng số vòng là D_L , thì $D_L = 2D_{\min}$.

Trên vòng L , chọn đường đơn dài nhất giữa hai lá liên tiếp rồi xóa nó đi; phần còn lại là một đường đi P . Vì có leaf đoạn giữa các lá, đoạn dài nhất có trọng số ít nhất D_L/leaf , do đó

$$D_P \leq \left(1 - \frac{1}{\text{leaf}}\right) D_L = 2 \left(1 - \frac{1}{\text{leaf}}\right) D_{\min}.$$

Trong G_1 , cạnh giữa hai đỉnh thuộc S luôn lấy đường đi ngắn nhất trong G , nên cây khung nhỏ nhất trên G_1 không nặng hơn đường P . Các bước khử trùng cạnh và xóa cạnh vô dụng sau đó không làm tăng tổng trọng số. Vì vậy

$$D_T \leq D_P < 2 \left(1 - \frac{1}{\text{leaf}}\right) D_{\min}.$$

Định lý 6.2. Khi $\text{leaf} > 2$, tỉ lệ xấu nhất đạt đúng

$$\frac{D_T}{D_{\min}} = 2 \left(1 - \frac{1}{\text{leaf}}\right).$$

Chứng minh. Dựng đồ thị $G = (V, E)$ như sau:

$$V = \{v_1, v_2, \dots, v_{\text{leaf}}, v_{\text{leaf}+1}\}, \quad S = \{v_1, v_2, \dots, v_{\text{leaf}}\}.$$

Đặt trọng số

$$d(v_i, v_j) = \begin{cases} 2, & j = i + 1, i < \text{leaf}, \\ 1, & i \leq \text{leaf}, j = \text{leaf} + 1, \\ \infty, & \text{các trường hợp khác.} \end{cases}$$

Trong trường hợp xấu nhất, thuật toán có thể chọn đường đi nối các đỉnh $v_1, v_2, \dots, v_{\text{leaf}}$ thành một chuỗi với tổng trọng số $2(\text{leaf} - 1)$. Trong khi đó, cây Steiner tối ưu là ngôi sao qua $v_{\text{leaf}+1}$, có tổng trọng số leaf . Do đó

$$\frac{D_T}{D_{\min}} = \frac{2(\text{leaf} - 1)}{\text{leaf}} = 2 \left(1 - \frac{1}{\text{leaf}}\right),$$

đúng bằng chặn trên.

4.6.4 Ví dụ 10: kiểm tra lẫn nhau đội tuyển 2015, *Marketing network*

Cho đồ thị $G = (V, E)$. Cần tìm K rừng sinh nhỏ nhất đầu tiên, với ràng buộc tập đỉnh đặc biệt S phải liên thông trong rừng sinh. Dữ liệu:

$$|V|, K < 50, \quad |E| < 100, \quad |S| < 15,$$

cạnh, trọng số cạnh và tập S đều được sinh ngẫu nhiên.

Thuật toán tối ưu. Bài toán nghiệm tốt thứ k có thể dùng A^* . Ta mô tả trạng thái bằng việc mỗi cạnh đã được chọn hay chưa; khi tìm kiếm, quyết định cạnh tiếp theo có được chọn hay không.

Nếu dùng thuật toán trạng thái nén cho cây Steiner làm hàm ước lượng, ta có $h(s) = h^*(s)$. Mỗi lần mở rộng trạng thái tốt nhất. Vì yêu cầu là rừng sinh, sau mỗi $O(n)$ quyết định hiệu lực có thể tìm được nghiệm tốt nhất hiện tại; tìm K nghiệm đầu chỉ cần $O(Km)$ lần tính ước lượng.

Thuật toán này bảo đảm tính đúng và độ phức tạp, nhưng hiệu suất thực tế thấp, không đủ cho miền dữ liệu của bài này.

Thuật toán xấp xỉ. Tiếp tục dùng tư tưởng A^* , nhưng lấy thuật toán heuristic ở phần trước làm ước lượng. Bất lợi là giá trị ước lượng có thể lớn hơn $h^*(s)$, nên không còn thể cất tĩa một cách chắc chắn.

Ta xử lý xấp xỉ bằng cách chọn một hằng số c , dùng $ch(s)$ làm tham khảo cho cận dưới, rồi cất các trạng thái có ước lượng kém. Điều chỉnh c cho phép cân bằng giữa tốc độ chạy và độ chính xác.

4.7 Lời cảm ơn

Xin cảm ơn Hiệp hội Máy tính Trung Quốc đã cung cấp nền tảng học tập và giao lưu.

Xin cảm ơn các thầy Chen Heli, Dong Yehua, Shao Hongxiang và You Guanghui của Trường Trung học số 1 Thiệu Hưng đã nhiều năm quan tâm và hướng dẫn tôi.

Xin cảm ơn các huấn luyện viên đội tuyển quốc gia Yu Linyun và Chen Xuji đã chỉ dẫn.

Xin cảm ơn các anh Yu Yili, Dong Anhua và He Qizheng của Đại học Thanh Hoa đã giúp đỡ tôi.

Xin cảm ơn các bạn Zhang Hengjie, Wang Jianhao và Jia Yuekai của Trường Trung học số 1 Thiệu Hưng đã giúp đỡ và gợi mở cho tôi.

Xin cảm ơn các thầy cô và bạn học khác từng giúp đỡ, gợi ý cho tôi.

Tài liệu tham khảo của bài viết

- [1] Wikipedia, Heuristic (computer science).
- [2] Wikipedia, Partition problem.
- [3] Wikipedia, Steiner tree problem.
- [4] L. Kou, G. Markowsky and L. Berman, “A fast algorithm for Steiner trees”, *Acta Informatica*.
- [5] Jiang Jinye, *Công cụ mạnh cho giải lập: tối ưu và ứng dụng thuật toán SPFA*, luận văn đội tuyển quốc gia 2009.
- [6] Zhou Erjin, *Bàn về ứng dụng hàm ước lượng trong thi tin học*, luận văn đội tuyển quốc gia 2009.
- [7] Yu Yili, *Một số phương pháp tìm nghiệm tốt thứ k*, luận văn đội tuyển quốc gia 2014.